Lecture 2: Computation — Julia and VFI

第二讲：计算 — Julia 与价值函数迭代

Computational Methods for Heterogeneous-Agent Macro
异质性主体宏观的计算方法

Jeffrey Sun

May 12, 2026

University of Toronto

Recap / 回顾

Bellman Equation
贝尔曼方程

$$V(b, w) = \max_{c \in [0, b]} u(c) + \beta \mathbb{E}\Big[V(R(b - c) + w', w') \,\Big|\, w \Big] \quad \forall b, w$$

This equation characterizes the function V that is needed to solve the household's problem. / 此方程刻画了求解家庭问题所需的函数 V。

V is Unique and Computable
V 唯一且可计算

For any bounded v_1, v_2,

$$\|\mathcal{T}v_1 - \mathcal{T}v_2\|_\infty \leq \beta \|v_1 - v_2\|_\infty .$$

Informally, the Banach fixed-point theorem implies:

- A unique V^* exists that satisfies $V^* = \mathcal{T}V^*$.
- Value function iteration will always find V^*.

对任意有界的 v_1, v_2：

非正式地，Banach 不动点定理意味着：

- 存在唯一的 V^* 满足 $V^* = \mathcal{T}V^*$。
- 价值函数迭代总能找到 V^*。

Today
今天

1. Replace the continuous state b with a finite grid.
2. Code $\mathcal{T}$ as a Julia function.
3. Iterate $v_{k+1} = \mathcal{T} v_k$ until $\|v_{k+1} - v_k\|_\infty < \text{tol}$.
4. Plot V.

1. 把连续状态 b 换成有限网格。
2. 用 Julia 写出 $\mathcal{T}$。
3. 迭代 $v_{k+1} = \mathcal{T} v_k$，直到 $\|v_{k+1} - v_k\|_\infty < \text{tol}$。
4. 画出 V。

Julia

Values and Types
值与类型

```
β = 0.96              # bound a value to a name
typeof(β)             # Float64
typeof(2)             # Int64
typeof("hello")       # String
```

Unicode names are valid. β is a Float64, 2 is an Int64.

Every value has a concrete **type**.

变量名可以用 Unicode。β 是 Float64，2 是 Int64。

每个值都有具体的**类型**。

Arrays
数组

```
xs  = [1.0, 2.0, 3.0]      # Vector{Float64}, length 3
M   = [1 2; 3 4]           # Matrix{Int64}, 2×2
zs  = zeros(3, 4)          # 3×4 matrix of 0.0
xs[1]                      # 1.0   --- 1-indexed!
M[2, 1]                    # 3
```

Indexing starts at 1.

`xs[end]` is the last element. `xs[2:3]` is a slice.

Arrays are column-major: dimension 1 is the row, dimension 2 is the column. A "vector" is a column vector.

下标从 1 开始。

`xs[end]` 是最后一个元素，`xs[2:3]` 是切片。

数组按列主序存储：第一维是行，第二维是列。所谓 "向量" 就是列向量。

Broadcasting
广播

```
xs = [1.0, 2.0, 3.0]

xs .+ 10          # [11.0, 12.0, 13.0]
log.(xs)          # [0.0, 0.693, 1.099]
xs .^ 2           # [1.0, 4.0, 9.0]

# elementwise on two arrays of the same shape:
[1,2,3] .* [10,20,30]   # [10, 40, 90]
```

The dot `.` is the "elementwise" marker. Dots apply any function or operation elementwise with no need for loops.

点号 `.` 是 "按元素" 标记，可以让任何函数逐元素作用——不用写循环。

Functions
函数

```
function utility(c)
    c <= 0 && return -Inf
    return log(c)
end

# shorthand for simple conditionals:
function utility(c)
    return c <= 0 ? -Inf : log(c)
end

# one-line functions equally valid:
u(c) = c <= 0 ? -Inf : log(c)
```

Discretization / 离散化问题

Discrete Wealth Grid
离散财富网格

V is a function on $[0, \infty) \times W$. We can't represent it exactly.

Pick N wealth values $b_1 < b_2 < \cdots < b_N$.

Represent V as the vector $[V(b_1), \ldots, V(b_N)]$.

This makes the function finite-dimensional.

V 是 $[0, \infty) \times W$ 上的函数，无法精确表示。

选 N 个财富点 $b_1 < b_2 < \cdots < b_N$。

将 V 表示为向量 $[V(b_1), \ldots, V(b_N)]$。

函数从此变成有限维对象。

Linear vs Log Spacing
线性等距 vs 对数等距

```
loggrid(lo, hi, N) = exp.(range(log(lo), log(hi); length=N))

# linear: equally spaced
b_grid = collect(range(0.1, 20.0; length=200))

# log: denser near the borrowing constraint
b_grid = loggrid(0.1, 20.0, 200)
```

The policy function is more nonlinear for small b, so we use log spacing to put more grid points on smaller b.

策略函数在 b 较小处更加非线性，因此采用对数等距，把更多网格点放在较小的 b 上。

Utility
效用

```
u(c) = c <= 0 ? -Inf : log(c)
```

Generalized version is CRRA with γ, but keep it simple for now.

推广形式是含 γ 的 CRRA，今天先保持简单。

Bellman Operator $\mathcal{T}$ as a Julia Function
贝尔曼算子 $\mathcal{T}$ 的 Julia 实现

```julia
function bellman_operator(V, params)
    (; β, R, w, b_grid) = params

    # end-of-period wealth needed to reach each b' on the grid
    b_end = (b_grid' .- w) ./ R

    # u(c) + β V(b') for every (b, b'); take max over b'
    return maximum(u.(b_grid .- b_end) .+ β .* V'; dims = 2)
end
```

`b_grid` is a column vector. `b_grid'` is a row vector. Each row corresponds to a b value; each column to a b' value. The last line takes `max` **within** each b (rows), **across** b' (columns).

`b_grid` 是列向量，`b_grid'` 是行向量。每一行对应一个 b 值，每一列对应一个 b' 值。最后一行在每个 b 内 **（行内）**、跨所有 b' **（列间）**取最大。

Value Function Iteration / 价值函数迭代

Value Function Iteration
价值函数迭代

1. Start with $V_0 \equiv 0$ (any initial guess works).
2. $V_{k+1} \leftarrow \mathcal{T} V_k$.
3. Stop when $\|V_{k+1} - V_k\|_\infty < \text{tol}$.

Banach theorem $\Rightarrow$ algorithm converges to true V^* at rate β.

1. 从 $V_0 \equiv 0$ 开始（任意初值都行）。
2. $V_{k+1} \leftarrow \mathcal{T} V_k$。
3. 当 $\|V_{k+1} - V_k\|_\infty < \text{tol}$ 时停。

由 Banach 不动点定理 $\Rightarrow$ 算法以速率 β 收敛到真实的 V^*。

solve_vfi / 求解函数

```
function solve_vfi(params; tol = 1e-6, maxiter = 2000, verbosity = 0)
    V = zeros(size(params.b_grid))

    Δ, iters = Inf, 0
    while Δ >= tol
        TV = bellman_operator(V, params)
        Δ  = maximum(abs.(TV .- V))
        V  = TV
        iters += 1
        iters > maxiter && error("VFI did not converge in $maxiter iterations")
    end

    verbosity >= 1 && println("VFI converged in $iters iterations with error $Δ")
    return V
end
```

Baseline Calibration
基准参数

```
b_grid = loggrid(0.05, 40.0, 400)
params = (; β = 0.96, R = 1.04, w = 1.0, b_grid)

V = solve_vfi(params; verbosity = 1)
```

$\beta R < 1$, otherwise V^* is infinite.

Grid: 400 log-spaced points from 0.05 to 40.

`params` is a `NamedTuple` (named tuple). The last `b_grid` is equivalent to `b_grid = b_grid`.

$\beta R < 1$，否则 V^* 无界。

网格：从 0.05 到 40 的 400 个对数等距点。

`params` 是一个 `NamedTuple`（具名元组）。末尾的 `b_grid` 等价于 `b_grid = b_grid`。

Features of Value Function
价值函数的性质

$V(b)$ is increasing because more wealth never hurts.

$V(b)$ is concave: an additional unit of wealth is worth more to the poor than to the rich.

The slope is the **marginal value of wealth**: $V'(b) = u'(c^\star(b))$ by the envelope theorem.

$V(b)$ 递增：财富越多绝不会更糟。

$V(b)$ 凹：多一单位财富对穷人比对富人更有价值。

斜率即**财富的边际价值**：由包络定理，$V'(b) = u'(c^\star(b))$。

Policy Function $c^\star$
策略函数 $c^\star$

$$c^\star(b) \;=\; \arg\max_{c \in [0,b]} \; u(c) + \beta\, V\big(R(b-c)+w\big).$$

```
function policy_function(V, params)
    (; β, R, w, b_grid) = params

    # same setup as bellman_operator
    b_end = (b_grid' .- w) ./ R

    # argmax over b' (columns) — return the chosen b'-index for each row
    idx = argmax(u.(b_grid .- b_end) .+ β .* V'; dims = 2)
    return vec(getindex.(idx, 2))::Vector{Int}
end
```

Same operator as `bellman_operator`; `argmax` in place of `maximum`. The return is a vector of integers: for each b, the grid-index of the optimal b'. / 与 `bellman_operator` 同一个算子；把 `maximum` 换成 `argmax`。返回一个整数向量：对每个 b，给出最优 b' 的网格下标。

Recap
回顾

V is now a **vector** we can examine, not just an abstract symbol. The policy $c^\star$ falls out by one $\arg\max$.

We have:

- a small Julia toolbox (values, arrays, broadcasting, functions)
- `bellman_operator` that codes $\mathcal{T}$ in a few broadcast lines
- `solve_vfi` that iterates to the fixed point
- `policy_function` that reads $c^\star$ off V

V 现在是一个可以查看的**向量**，不再只是抽象符号。策略 $c^\star$ 只差一次 $\arg\max$。

我们已经有：

- 一个小型 Julia 工具集（值、数组、广播、函数）
- 用几行广播实现 $\mathcal{T}$ 的 `bellman_operator`
- 迭代到不动点的 `solve_vfi`
- 从 V 读出 $c^\star$ 的 `policy_function`

Tomorrow: Heterogeneity
明天：异质性

Add an income shock, state grows from b to (b, w).

Heterogeneity: We can think of each (b, w) state as a different **type** of households
Different b: rich vs. poor
Different w: high vs. low

加入一个收入冲击，状态从 b 变为 (b, w)。

异质性：可以把每个 (b, w) 状态看作一种不同**类型**的家庭。
不同的 b：富 vs. 穷
不同的 w：高 vs. 低